



Universidad Experimental Nacional de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: INGENIERÍA DE SOFTWARE II

Ensayo crítico: ventajas y desventajas (PMBOK 7 → PMBOK 8) para la Ingeniería de Software

Profesor:

Marquez Felix

Estudiantes:

Alondra Moreno CI: 30.331.870

Ciudad Guayana, Septiembre 2026

A continuación se adjunta el prompt diseñado bajo el rol de Senior Project Manager para realizar la comparativa académica requerida:

"Actúa como un Senior Project Manager y experto académico en metodologías de Ingeniería de Software. Necesito que realices un análisis comparativo riguroso y exhaustivo entre el PMBOK 7ma Edición (2021) y las actualizaciones recientes o proyectadas del PMBOK 8va Edición (2026).

Por favor, estructura tu respuesta de la siguiente manera:

1. Evolución Filosófica: ¿Cuál es el cambio de paradigma principal entre la versión 7 y la 8?
2. Comparativa Técnica: Una tabla o lista detallada contrastando los dominios, principios o áreas de conocimiento que se añadieron, eliminaron o transformaron.
3. Impacto en Ingeniería de Software: Un análisis crítico de las ventajas y desventajas que esta nueva edición supone específicamente para la gestión de proyectos de desarrollo de software y productos digitales.

El tono debe ser estrictamente académico, objetivo y crítico. Utiliza terminología profesional de gestión de proyectos y desarrollo de software."

Las LLms utilizadas para la Realización de este ensayo fueron: Gemini y Chatgpt

Lo que encontré al comparar ambos LLMs

1) Cambios principales (¿coinciden o proponen cambios distintos?)

Coinciden en lo esencial. Los dos describen el mismo cambio macro:

PMBOK 7 = enfoque más **abstracto** (principios/dominos, valor, *tailoring*).

PMBOK 8 = mantiene la base de principios pero **reincorpora estructura operativa/procesual** para hacerlo más “práctico/accionable”, sin volver a un modelo rígido tradicional.

Diferencia: **Gemini** lo explica como “más aplicabilidad” y reintroducción “no prescriptiva”;

Chatgpt lo formula como “síntesis híbrida pragmática”.

2) Interpretación (¿entienden igual o uno lo enfoca diferente?)

Ambos entienden la pregunta igual (comparar filosofía, cambios técnicos e impacto en software).

Pero el **enfoque es distinto**:

Gemini: más **técnico-estructural/academicista**, organiza por secciones, listas, comparativas y “qué cambió” con detalle.

Chatgpt: más **estratégico-discursivo**, usa un marco conceptual (“madurez contextual”, “hibridación pragmática”) y lo conecta con realidades organizacionales (PMO, contratos, enterprise).

3) IA y sostenibilidad (¿las mencionan y las conectan?)

IA: sí, ambos la mencionan explícitamente.

Gemini la incluye como un “nuevo énfasis” y la relaciona con gobernanza, riesgos, ética, IP y uso de IA como herramienta de gestión.

Chatgpt la plantea como integración transversal en todo el ciclo de vida (analítica predictiva, IA generativa, soporte a decisiones).

Sostenibilidad: no ambos.

Gemini sí la menciona explícitamente (la integra como principio/tema).

Chatgpt no la desarrolla en el texto compartido.

Conexión IA–sostenibilidad: ninguno las conecta de forma clara (Gemini las menciona, pero no articula una relación directa entre IA y sostenibilidad; **Chatgpt** directamente no trata sostenibilidad).

4) Detalle de procesos (¿quién describe mejor el “cómo”?)

Gemini describe mejor el “cómo”. Da más contenido operativo: estructura (principios/dominos/focus areas), reintegración de áreas de proceso, ejemplos de aplicación en software (híbrido, DevOps), y componentes como gobernanza/finanzas/calidad integradas.

Chatgpt aporta ideas aplicadas (p. ej. riesgos de burocratización, contratos, PMO, IA para estimación/riesgos/deuda técnica), pero en general es **menos paso-a-paso** y más de “marco” que de procedimiento.

En los proyectos de software, el reto casi nunca es “seguir un método correcto”, sino equilibrar dos necesidades que compiten todo el tiempo: avanzar rápido y adaptarse a cambios, pero sin perder el control del presupuesto, los riesgos, la calidad y los compromisos con clientes o con la organización. En ese contexto, el paso de un enfoque más “de principios” (PMBOK 7) a uno que busca ser más “aplicable” (lo que se suele atribuir a PMBOK 8) puede traer beneficios importantes, aunque también riesgos si se usa mal.

Ventajas: por qué puede ayudar en proyectos de software

1) Da más claridad sobre cómo trabajar, sin quitar flexibilidad

Una crítica frecuente a los enfoques muy generales es que inspiran, pero no guían. En software, donde las decisiones diarias son constantes, tener una estructura más clara puede ayudar a equipos y líderes a organizarse mejor: qué revisar, qué decidir primero, cómo asegurar seguimiento, etc.

La ventaja aparece cuando esa estructura se usa como orientación, no como una receta rígida.

2) Facilita el diálogo con personas no técnicas

Muchos proyectos fallan no por el código, sino por expectativas confusas: qué se promete, cuándo, con qué alcance y con qué costo. Un marco más “ordenado” puede ayudar a traducir el trabajo del equipo a un lenguaje que negocio, finanzas o dirección entienden mejor.

Eso reduce malentendidos y mejora la toma de decisiones (por ejemplo, qué vale la pena construir primero y qué no).

3) Refuerza la idea de “valor”, no solo “entregas”

En software es muy fácil caer en la trampa de medir “cuántas cosas se hicieron” en lugar de “qué cambió realmente para el usuario o para el negocio”. Un enfoque que insiste en valor empuja preguntas sanas:

¿esto mejora la experiencia?

¿reduce errores?

¿ahorra tiempo o dinero?

¿mejora la confianza y la estabilidad?

4) Permite combinar estilos de trabajo sin peleas

En la práctica, muchos equipos trabajan con mezcla: partes muy planificadas (por ejemplo, temas legales, compras, seguridad) y otras iterativas (diseño y construcción del producto). Si el marco reconoce esa mezcla como normal, se reduce la presión de “todo debe ser ágil” o “todo debe estar definido desde el inicio”.

5) Abre espacio para hablar con seriedad de IA (sin idealizarla)

La IA puede mejorar productividad, pero también trae riesgos (errores, uso indebido de datos, dependencia, problemas legales). Que se incluya como tema obliga a tratarla de forma más responsable: no como moda, sino como algo que requiere criterios, supervisión y límites.

Desventajas: cómo puede perjudicar a la ingeniería de software

1) Riesgo de volver a burocracia y controles excesivos

El peligro más común cuando se “reintroduce estructura” es que algunas organizaciones lo interpreten como: más reportes, más aprobaciones, más reuniones, más plantillas.

Eso puede frenar a los equipos y hacerlos trabajar para el sistema de control en vez de para el producto.

2) Puede alimentar el “cumplir el plan” por encima de “aprender y ajustar”

En software, cambiar de rumbo a tiempo puede ser una virtud, no un fracaso. Si la gestión se vuelve demasiado rígida se castiga el ajuste y se premia aparentar que “todo va según lo planeado”, aunque el producto esté yendo hacia un lugar equivocado.

3) No resuelve por sí solo los problemas de calidad

Un marco de gestión ayuda a organizar, pero no sustituye prácticas esenciales del equipo: buena comunicación, revisión de trabajo, pruebas, disciplina, aprendizaje. Si una organización cree que con adoptar un estándar “ya está”, puede descuidar lo que realmente sostiene un software confiable.

4) Puede ser demasiado pesado para equipos pequeños o startups

Cuando el equipo es reducido y el entorno cambia rápido, una capa grande de gestión puede costar más de lo que aporta. Ahí lo que suele funcionar mejor es una estructura mínima: objetivos claros, prioridades, feedback constante y responsabilidad directa.

Conclusión

Para la ingeniería de software, el valor de un enfoque más “aplicable” está en ayudar a ordenar decisiones y dar visibilidad sin destruir la flexibilidad. Su principal riesgo es convertirse en un pretexto para control excesivo y burocracia. En resumen puede ser un puente útil entre ingeniería y negocio, siempre que se use como guía adaptable y no como un conjunto de reglas que se siguen “porque sí”.